

Guía Explicativa: Flex & Bison (Ejercicios 1.1 al 1.6)

Análisis conceptual, desglose de código, flujo de datos y relación entre analizador léxico (Flex) y sintáctico (Bison).

1. Fundamentos: Análisis Léxico vs. Análisis Sintáctico

El procesamiento de lenguajes formales se divide en dos fases fundamentales:

- Análisis Léxico (Flex - Scanner):** Lee el flujo de caracteres y lo divide en unidades mínimas con significado llamadas **tokens** (números, operadores, identificadores).
- Análisis Sintáctico (Bison - Parser):** Recibe los tokens del escáner y verifica que cumplan con la gramática formal (BNF), evaluando expresiones o construyendo el árbol de derivación.

Estructura de 3 secciones (%{ %} / %% / %%): (1) Declaraciones C y tokens, (2) Reglas y acciones, y (3) Rutinas de usuario en C.

2. Ejercicios Aislados (1.1 al 1.4: Solo Flex)

1.1: Contador de Palabras (fb1-1.1) AISLADO

Propósito: Simular el comando `wc` de Unix contando caracteres, palabras y líneas.

```
%{
int chars = 0, words = 0, lines = 0;
}%
%option noyywrap
%%
[a-zA-Z]+ { words++; chars += strlen(yytext); }
\n      { chars++; lines++; }
.       { chars++; }
%%
int main() {
    yylex();
    printf("%8d%8d%8d\n", lines, words, chars);
    return 0;
}
```

- `[a-zA-Z]+` : Secuencia de letras. Suma palabra y longitud.
- `\n` : Salto de línea (incrementa líneas y caracteres).
- `.` : Regla comodín para cualquier otro carácter.
- `yylex()` : Bucle principal de escaneo hasta EOF.

1.2: Traductor Léxico (fb1-2.1) AISLADO

Propósito: Reemplazo directo de patrones inglés a americano.

```
%option noyywrap
%%
"colour"      { printf("color"); }
"flavour"     { printf("flavor"); }
"clever"      { printf("smart"); }
"smart"       { printf("elegant"); }
"conservative" { printf("liberal"); }
.             { printf("%s", yytext); }
%%
int main() { yylex(); return 0; }
```

- Reglas de Flex:** Prioriza la coincidencia más larga; ante empate, la primera regla en el archivo.
- `.` : Reenvía cualquier carácter no listado intacto.

1.3: Escáner Simple (fb1-3.l)

AISLADO

Propósito: Mostrar nombres textuales de tokens reconocidos.

```
%option noyywrap
%%
"+" { printf("PLUS\n"); }
"_" { printf("MINUS\n"); }
"*" { printf("TIMES\n"); }
"/" { printf("DIVIDE\n"); }
"|" { printf("ABS\n"); }
[0-9]+ { printf("NUMBER %s\n", yytext); }
\n { printf("NEWLINE\n"); }
[ \t] { /* ignora espacios */ }
. { printf("Mystery %s\n", yytext); }
%%
```

- Reconoce operadores como tokens textuales.
- `[ \t]` con bloque vacío descarta espacios en blanco.
- `.` captura caracteres no reconocidos.

1.4: Tokens y Variable yylval (fb1-4.l)

AISLADO

Propósito: Retornar códigos enteros y pasar valor numérico vía `yylval`.

```
{
enum yytokentype {
    NUMBER = 258, ADD = 259, SUB = 260,
    MUL = 261, DIV = 262, ABS = 263, EOL = 264
};
int yyval;
}
%option noyywrap
%%
"+" { return ADD; }
"_" { return SUB; }
"*" { return MUL; }
"/" { return DIV; }
"|" { return ABS; }
[0-9]+ { yyval = atoi(yytext); return NUMBER; }
\n { return EOL; }
[ \t] { }
. { printf("Mystery %c\n", *yytext); }
%%
int main() {
    int tok;
    while((tok = yylex())) {
        if(tok == NUMBER) printf("%d = %d\n", tok, yyval);
        else printf("%d\n", tok);
    }
}
```

- `return TOKEN :` `yylex()` pausa y entrega el token.
- `≥ 258 :` Códigos seguros que no colisionan con ASCII.

3. Ejercicios Complementarios (1.5 y 1.6: Flex + Bison en Conjunto)

¿Por qué 1.5 y 1.6 forman un solo programa? El **Ejercicio 1.5 (fb1-5.y)** define la gramática BNF y las operaciones matemáticas en Bison. El **Ejercicio 1.6 (fb1-5.l)** es el escáner léxico en Flex adaptado para conectarse con Bison mediante el archivo de cabecera generado `fb1-5.tab.h`. Se compilan juntos en un único ejecutable.

1.5: Parser en Bison (fb1-5.y)

COMPLEMENTARIO

Propósito: Reglas gramaticales BNF y jerarquía de precedencia.

```
%{
#include <stdio.h>
int yylex(void);
void yyerror(char *s);
}%
%token NUMBER ADD SUB MUL DIV ABS EOL
%%
calclist: /* vacío */
        | calclist exp EOL { printf("= %d\n", $2); }
        ;
exp: factor
    | exp ADD factor { $$ = $1 + $3; }
    | exp SUB factor { $$ = $1 - $3; }
    ;
factor: term
      | factor MUL term { $$ = $1 * $3; }
      | factor DIV term { $$ = $1 / $3; }
      ;
term: NUMBER
     | ABS term { $$ = $2 >= 0 ? $2 : -$2; }
     ;
%%
int main() { yyparse(); return 0; }
void yyerror(char *s) { fprintf(stderr, "error: %s\n", s); }
```

- **Precedencia por capas:** `term` (`|x|`, num) > `factor` (`*`, `/`) > `exp` (`+`, `-`).
- `$$` : Valor resultante de la regla actual.
- `$1`, `$2`, `$3` : Valores de los elementos del lado derecho.
- `yyparse()` : Solicita tokens a `yylex()` y evalúa.

1.6: Escáner para Bison (fb1-5.l)

COMPLEMENTARIO

Propósito: Proveedor de tokens para el parser de Bison.

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "fb1-5.tab.h" /* Header de Bison */
}%
%option noyywrap
%%
"+" { return ADD; }
"-" { return SUB; }
"*" { return MUL; }
"/" { return DIV; }
"|" { return ABS; }
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
\n { return EOL; }
[ \t] { /* ignora espacios */ }
. { printf("Mystery %c\n", *yytext); }
%%
```

- **Integración:** Incluye `fb1-5.tab.h` (Bison define los tokens y `yylval`).
- **Sin `main()`** : El punto de entrada está en Bison.

4. Cuadro Comparativo Global

Ej.	Archivo	Herramienta	Tipo	Descripción Funcional
1.1	fb1-1.1	Flex	Aislado	Contador de caracteres, palabras y líneas (estilo <code>wc</code>).
1.2	fb1-2.1	Flex	Aislado	Traductor léxico inglés británico a inglés americano.
1.3	fb1-3.1	Flex	Aislado	Escáner que imprime por pantalla los nombres de los tokens.
1.4	fb1-4.1	Flex	Aislado	Escáner con <code>enum</code> y asignación semántica a <code>yylval</code> .
1.5	fb1-5.y	Bison	Complementario	Gramática BNF, precedencia de operadores y cálculo de la calculadora.
1.6	fb1-5.1	Flex	Complementario	Escáner conectado a Bison mediante el archivo <code>fb1-5.tab.h</code> .

5. Comandos de Compilación y Ejecución

Ejercicios Aislados (1.1 a 1.4):

```
flex fb1-1.1
gcc lex.yy.c -o fb1-1
./fb1-1
```

Sistema Integrado (1.5 + 1.6):

```
# 1. Bison genera parser y header
bison -d fb1-5.y
# 2. Flex genera escáner usando fb1-5.tab.h
flex fb1-5.1
# 3. Compilación y enlace conjunto
gcc fb1-5.tab.c lex.yy.c -o fb1-5
# 4. Prueba: 20 - 4 / 2 -> = 18
./fb1-5
```